

Object Localization

Overview

This week focuses on **Object Detection**, a rapidly advancing area of computer vision. To understand detection, we first define **Object Localization**.

- **Image Classification:** Determines *what* is in the image (e.g., "This is a car").
- **Classification with Localization:** Determines *what* is in the image and *where* it is by outputting a **bounding box**. (Usually assumes one major object).
- **Object Detection:** Finds and localizes **multiple** objects of potentially different categories within a single image.

Defining the Bounding Box

To localize an object, the neural network is modified to output four additional numbers alongside the class label.

Coordinates

- The bounding box is parameterized by four numbers:
 - b_x, b_y : The coordinates of the **midpoint** (center) of the object.
 - b_h : The **height** of the bounding box.
 - b_w : The **width** of the bounding box.
- **Coordinate System:**
 - Top-left corner: $(0, 0)$
 - Bottom-right corner: $(1, 1)$
- **Example:** A box roughly in the center might have $b_x \approx 0.5, b_y \approx 0.5$.

Target Label Encoding (y)

For supervised learning, the target label y is a vector that encodes both the presence of an object, its location, and its class.

The Vector Structure

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

- p_c (**Probability of Content**): Indicates if an object is present.
 - 1: Object is present (Pedestrian, Car, or Motorcycle).
 - 0: Background (No object).
- b_x, b_y, b_h, b_w : Bounding box coordinates.
- c_1, c_2, c_3 : Class indicators (e.g., Pedestrian, Car, Motorcycle).

Examples

1. Object Present (e.g., Car - Class 2):

$$y = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

2. No Object (Background):

$$y = \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

- Note: The "?" represents "don't care" values. If $p_c = 0$, the loss function ignores the bounding box and class label components.

Loss Function

The loss function adapts based on whether an object is present ($y_1 = 1$) or not ($y_1 = 0$).

- **Case 1: Object Present ($y_1 = 1$):**

- The loss is calculated on **all** components (p_c , bounding box, and class labels).
- *Simplified Example (Squared Error):*

$$\mathcal{L}(\hat{y}, y) = (\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + \dots + (\hat{y}_8 - y_8)^2$$

- **Case 2: Background ($y_1 = 0$):**

- The loss is calculated **only** on the p_c component. The other components are ignored.

$$\mathcal{L}(\hat{y}, y) = (\hat{y}_1 - y_1)^2$$

- *Implementation Note:* While squared error is used here for simplicity, typically:

- **Log-likelihood/Softmax loss** is used for class labels ($c_1 \dots c_3$).
 - **Squared error** is used for bounding box coordinates ($b_x \dots b_w$).
 - **Logistic regression loss** is used for p_c .
-

Landmark Detection

Concept

Building upon object localization (which outputs a bounding box), neural networks can be trained to output specific X and Y coordinates for important points within an image, known as **landmarks**.

- **Generalization:** Instead of just 4 numbers (b_x, b_y, b_h, b_w), the network outputs a sequence of coordinates $(l_{1x}, l_{1y}), (l_{2x}, l_{2y}), \dots$ corresponding to specific feature points.

Applications

1. Face Recognition & Augmented Reality (AR)

To detect facial features, emotions, or apply filters (e.g., Snapchat lenses), the network localizes specific points on the face.

- **Landmarks:** Corners of eyes, corners of the mouth, edges of the nose, and the

jawline.

- **Architecture Example:**

- **Input:** Image of a face.
- **Goal:** Detect 64 unique landmarks.
- **Output Layer:** A vector containing:
 1. p_c (Is there a face? 0 or 1).
 2. l_{1x}, l_{1y} (Coordinates of landmark 1).
 3. ... up to l_{64x}, l_{64y} .
- **Total Output Units:** $1 + (64 \times 2) = 129$ units.

- **Use Cases:**

- **Emotion Recognition:** Analyzing mouth curvature (smile vs. frown).
- **Computer Graphics:** Warping faces or placing digital objects (crowns, hats) correctly on the head.

2. Pose Detection

To determine a person's physical stance or pose.

- **Landmarks:** Key body joints and points (midpoint of chest, left shoulder, left elbow, wrist, etc.).
- **Architecture:** Similar to face detection, the network outputs (x, y) coordinates for a predefined set of body parts (e.g., 32 key points).

Data Labeling Requirements

For landmark detection to work, the training data must be consistently labeled.

- **Consistency:** The identity of each landmark must be constant across all images.
 - *Example:* "Landmark 1" must **always** be the left corner of the left eye in every single training image. "Landmark 2" must always be the right corner, etc.
 - **Labor Intensity:** Creating these datasets requires manual annotation of potentially dozens of points per image.
-

Object Detection: Sliding Windows Algorithm

Concept

Building upon object localization, **Object Detection** involves identifying and localizing multiple objects within an image. One of the simplest methods to achieve this is the **Sliding Windows Detection Algorithm**.

1. Training Phase

Before performing detection, a Convolutional Neural Network (ConvNet) must be trained to recognize the specific object (e.g., a car).

- **Training Set:**
 - **Input (x):** Closely cropped images of the object. The object should be centered and occupy most of the image frame.
 - **Labels (y):** Binary labels (0 or 1).
 - 1: Contains the object (Car).
 - 0: Does not contain the object (Background/No Car).
- **Goal:** Train the ConvNet to output $y \in \{0, 1\}$ given a cropped input.

2. The Sliding Window Algorithm (Inference)

Once the ConvNet is trained, it is used to detect objects in a full test image.

The Process

1. **Select Window Size:** Pick a specific rectangular window size.
2. **Input Region:** Extract the image region defined by the window and resize it to the ConvNet's input dimensions.
3. **Predict:** Feed the region into the ConvNet to get a prediction (0 or 1).
4. **Slide (Stride):** Shift the window by a fixed step size (stride) to a new position and repeat the prediction.
5. **Iterate:** Continue until every region of the image has been classified.
6. **Multi-Scale:** Repeat steps 1–5 with **larger window sizes** to detect objects that appear larger in the image.

3. Disadvantages

The primary drawback of the Sliding Windows algorithm is **Computational Cost**.

The Stride Trade-off

- **Small Stride:**
 - *Pro:* High granularity; less likely to miss an object or have poor localization.
 - *Con:* Requires processing a massive number of windows. Since modern ConvNets are computationally expensive, this is often infeasible slow.
- **Large Stride:**
 - *Pro:* Fewer windows to process; faster.
 - *Con:* Coarse granularity; may hurt performance or fail to accurately localize objects centered between stride steps.

Historical Context

Before Deep Learning, sliding windows were used with simple linear classifiers (over hand-engineered features). Because linear classifiers were cheap to compute, the computational cost was manageable. However, running a deep ConvNet hundreds or thousands of times for a single image is too slow for real-time applications.

Convolutional Implementation of Sliding Windows

Motivation

The standard Sliding Windows algorithm is computationally expensive because it requires running a full forward propagation of the ConvNet for every cropped region independently. This results in significant redundant computation, as many cropped regions overlap.

To solve this, we can implement the sliding windows algorithm **convolutionally**, allowing the entire image to be processed in a single forward pass.

1. Converting Fully Connected Layers to Convolutional Layers

The first step is to modify the network architecture by replacing Fully Connected (FC) layers with Convolutional layers.

Architecture Transformation Example

Consider a standard object detection ConvNet:

1. **Input:** $14 \times 14 \times 3$ image.
2. **Conv 1:** 5×5 filters (16 count) $\rightarrow$ Output: $10 \times 10 \times 16$.
3. **Max Pool:** $2 \times 2 \rightarrow$ Output: $5 \times 5 \times 16$.
4. **FC 1:** 400 units (Standard connection to the flattened pool output).
5. **FC 2:** 400 units.
6. **Softmax:** 4 outputs ($1 \times 1 \times 4$).

The Conversion:

Instead of flattening the $5 \times 5 \times 16$ volume to a vector:

- **Replace FC 1:** Use **400 filters of size $5 \times 5 \times 16$** .
 - Convolving a $5 \times 5 \times 16$ input with a matching filter yields a 1×1 output.
 - With 400 filters, the output volume becomes $1 \times 1 \times 400$.
- **Replace FC 2:** Use **400 filters of size 1×1** .
 - Output becomes $1 \times 1 \times 400$.
- **Replace Softmax:** Use **4 filters of size 1×1** .
 - Output becomes $1 \times 1 \times 4$.

Mathematically, this is identical to the fully connected version, but the network is now fully convolutional.

2. Implementing Sliding Windows Convolutionally

Once the network is fully convolutional, it can accept input images larger than the training size (14×14).

Mechanism

Based on the **OverFeat paper** (Sermanet et al.), applying the network to a larger image automatically performs the sliding window operation.

Example:

- **Training Input:** $14 \times 14 \times 3 \rightarrow$ **Output:** $1 \times 1 \times 4$.
- **Test Input:** $16 \times 16 \times 3$ (Larger image).
 - Instead of cropping the test image into four overlapping 14×14 squares and running the network four times:
 - Run the **entire** 16×16 **image** through the "conv-ified" network.
 - **Result:** The final output volume will be $2 \times 2 \times 4$.

Interpretation of the Output

Each cell in the $2 \times 2 \times 4$ output corresponds to a specific window position:

- **Top-Left cell:** Result of the sliding window at the top-left of the image.
- **Top-Right cell:** Result of the sliding window shifted by the stride (2 pixels).
- **Bottom-Left/Right cells:** Results of the lower windows.

Benefit

- **Computation Sharing:** Instead of re-computing the early activation layers for every overlapping crop, the convolutional implementation computes them once for the whole image. The shared regions (e.g., the overlap between the windows) share the computation naturally.
- **Efficiency:** This allows for simultaneous prediction of all sliding window positions in a **single forward pass**, making the process significantly faster.

Limitation

While efficient, this method still relies on the fixed positions and strides of the sliding windows. The bounding boxes might not perfectly align with the object (e.g., if the object is between strides or has a different aspect ratio).

Bounding Box Predictions (YOLO Algorithm)

The Problem with Sliding Windows

While the convolutional implementation of sliding windows is computationally efficient, it suffers from **localization inaccuracy**:

- **Discretization:** The bounding boxes are determined by the stride and window size, meaning they rarely match the object's actual position perfectly.
- **Aspect Ratio:** Sliding windows typically use square boxes, but objects (like cars) are often rectangular.

The YOLO Algorithm (You Only Look Once)

Proposed by Redmon et al., YOLO solves the accuracy problem by combining object detection into a single convolutional network pass that outputs precise bounding boxes.

1. Grid Division

- **Method:** Overlay a grid on the input image (e.g., 3×3). In practice, finer grids like 19×19 are used.
- **Concept:** Apply the image classification and localization algorithm (from the first video) to **each** grid cell simultaneously.

2. Object Assignment Rule

- **Midpoint Rule:** An object is assigned to a specific grid cell if and only if the **midpoint (center)** of the object falls within that grid cell.
- **Exclusivity:** Even if an object spans multiple grid cells, it is assigned to **only one** cell (the one containing the midpoint).

3. Output Vectors and Volume

For each grid cell, the network outputs a label vector y . Assuming 3 classes (Pedestrian, Car, Motorcycle):

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

- **Target Output Dimension:** For a 3×3 grid and an 8-dimensional vector (5 for box + 3 classes), the output volume is:
 $3 \times 3 \times 8$
- **Input:** $100 \times 100 \times 3$ image.
- **Mapping:** The ConvNet maps the input image directly to this $3 \times 3 \times 8$ output volume in a single forward pass.

4. Encoding Bounding Boxes

The bounding box parameters (b_x, b_y, b_h, b_w) are specified relative to the **grid cell**, not the entire image.

- **Coordinate System:**
 - Top-left of the grid cell: (0, 0).
 - Bottom-right of the grid cell: (1, 1).
- b_x, b_y (**Midpoint**):
 - Represents the position of the center point relative to the cell bounds.
 - **Constraint:** Must be between 0 and 1 (since the midpoint is, by definition, inside the cell).
- b_h, b_w (**Dimensions**):
 - Represents the height and width of the object relative to the grid cell's dimensions.
 - **Note:** These can be **greater than 1** if the object is larger than the grid cell itself (which is common).

5. Advantages

- **Accuracy:** Outputs explicit bounding box coordinates, allowing for precise localization and arbitrary aspect ratios.
- **Speed:** It is a **Convolutional Implementation**. The network does not run the algorithm 9 times (or 361 times for a 19×19 grid); it runs once. This shared computation makes YOLO extremely fast, enabling **real-time object detection**.

Note on Literature

The YOLO paper is known for being difficult to read and understand, even for senior researchers. It involves complex parameterizations (e.g., sigmoid functions for constraints, exponential parameterizations for width/height) that are often better understood by examining open-source code.

Intersection Over Union (IoU)

Purpose

Intersection Over Union (IoU) is a metric used to evaluate the accuracy of an object detection algorithm. It measures the overlap between two bounding boxes:

1. **Ground-Truth Bounding Box:** The actual labeled position of the object.
2. **Predicted Bounding Box:** The output from the algorithm.

Calculation

IoU is defined as the ratio of the area of the intersection to the area of the union of the two boxes.

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

- **Intersection:** The area where the two boxes overlap (the smaller region).
- **Union:** The total area covered by both boxes combined (the larger region).

Evaluating Correctness

IoU maps the concept of "localization" to a quantifiable accuracy score.

- **Convention:** A detection is typically considered **correct** if $\text{IoU} \geq 0.5$.
 - If $\text{IoU} = 1$: Perfect overlap (The predicted box is identical to the ground truth).
 - If $\text{IoU} < 0.5$: Generally considered a bad or incorrect detection.
- **Stringency:** The threshold of 0.5 is a human-chosen convention. For stricter evaluations (e.g., in competitions), higher thresholds like 0.6 or 0.7 may be used. It is rare to see a threshold below 0.5.

General Utility

Beyond evaluation, IoU serves as a general measure of **similarity between two bounding boxes**. This property is crucial for algorithms like **Non-Max Suppression**, which will be discussed in the next video.

Non-Max Suppression

The Problem: Multiple Detections

In object detection algorithms (like YOLO), a grid is placed over the image (e.g., 19×19). While an object technically belongs to only one grid cell (the one containing its midpoint), in practice, adjacent cells often also detect the object with high probability.

- **Result:** The algorithm outputs multiple bounding boxes for the same object.
- **Goal:** Clean up these redundant detections so that each object is detected exactly once.

The Solution: Non-Max Suppression (NMS) Algorithm

The algorithm suppresses (discards) "non-maximal" probabilities—meaning it keeps the best bounding box and removes significantly overlapping boxes that likely refer to the same object.

How it Works (Conceptual Steps)

1. **Filter by Threshold:** Discard all bounding boxes with a probability score (p_c) below a certain threshold (e.g., < 0.6).
2. **Select Best Box:** Pick the remaining box with the **highest** probability. Output this as a prediction (highlight it).
3. **Suppress Overlaps:** Discard any remaining box that has a high **Intersection Over Union (IoU)** (e.g., $\text{IoU} \geq 0.5$) with the box selected in the previous step.
 - *Logic:* If a box overlaps heavily with the "best" box, it is likely detecting the same object redundantly.
4. **Repeat:** Loop through the remaining boxes, repeatedly picking the highest probability box and suppressing its neighbors, until no boxes remain unprocessed.

Algorithm Steps (Formal)

For a specific class (e.g., "Car"):

1. **Input:** A list of predicted bounding boxes, each with a probability p_c and coordinates.
2. **Discard** all boxes where $p_c \leq 0.6$.
3. **While** there are any remaining boxes:
 - **Pick** the box with the largest p_c . Output this as a prediction.
 - **Discard** any remaining box with $\text{IoU} \geq 0.5$ (overlap) with the box output in the current step.

Handling Multiple Classes

If the network detects multiple classes (e.g., Pedestrians, Cars, Motorcycles):

- **Independent Execution:** Run the Non-Max Suppression algorithm independently for **each class**.
 - Run once for all "Car" detections.
 - Run again for all "Pedestrian" detections.
 - Run again for all "Motorcycle" detections.
-

Anchor Boxes

The Problem: Overlapping Objects

In the basic object detection algorithm, each grid cell can detect only **one** object.

- **Scenario:** What if two objects (e.g., a pedestrian and a car) have their midpoints in the *same* grid cell?
- **Limitation:** The standard output vector y can only output one set of bounding box coordinates and one class. It cannot handle the collision.

The Solution: Anchor Boxes

Anchor boxes allow a single grid cell to detect multiple objects by pre-defining a set of bounding box shapes (e.g., a tall, narrow box for pedestrians and a wide, short box for cars).

How It Works

Instead of the output vector y describing just *one* object, it is expanded to describe *multiple* objects, each associated with a specific anchor box shape.

Example with 2 Anchor Boxes:

- **Anchor Box 1:** Tall and narrow (Shape of a pedestrian).
- **Anchor Box 2:** Wide and short (Shape of a car).

New Output Vector y :

The vector now contains two stacked sets of the original outputs:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

← Associated with Anchor Box 1

← Associated with Anchor Box 2

- **Dimension:** For a 3×3 grid and 2 anchor boxes (assuming 3 classes), the output volume becomes $3 \times 3 \times 16$ (since each anchor box needs 8 outputs: 5 for the box + 3 for classes).
- *General Formula:* Grid Size $\times$ Grid Size $\times$ (Number of Anchors $\times$ (5 + Number of Classes)).

Assignment Rule

Previously, an object was assigned to the grid cell containing its midpoint. Now, an object is assigned to:

1. The grid cell containing its midpoint.
2. The **Anchor Box** that has the highest **IoU (Intersection Over Union)** with the

object's ground truth shape.

Example Scenario:

- A grid cell contains the midpoint of a pedestrian and a car.
- **Pedestrian:** Higher IoU with Anchor Box 1 (Tall).
 - Assigned to the top half of the y vector for that cell.
- **Car:** Higher IoU with Anchor Box 2 (Wide).
 - Assigned to the bottom half of the y vector for that cell.

Handling Edge Cases

- **Two objects, same grid, same anchor shape:** The algorithm cannot handle this perfectly. A default tie-breaker is used.
- **More objects than anchor boxes:** If 3 objects appear in one cell but you only have 2 anchor boxes, the algorithm cannot detect all three.
- *Note:* In practice, with finer grids (e.g., 19×19), these collisions are extremely rare.

Benefits Beyond Collisions

While motivated by overlapping objects, the primary benefit of anchor boxes is actually **specialization**.

- It allows the learning algorithm to "specialize" different parts of the neural network.
- One set of outputs learns to become experts at detecting tall objects (pedestrians), while another set specializes in wide objects (cars).

Choosing Anchor Boxes

1. **Hand-Picked:** Manually select 5-10 shapes that cover the variety of objects you expect to detect (e.g., some tall, some wide, some square).
 2. **K-Means Clustering (Advanced):** Run K-Means on the bounding box shapes in your training set to automatically group them and find the most representative "centroid" shapes. This is used in later versions of YOLO (YOLOv2).
-

Putting It Together: The YOLO Algorithm

This video synthesizes all previous concepts—Grid Cells, Anchor Boxes, Intersection Over Union (IoU), and Non-Max Suppression—into the complete YOLO (You Only Look Once) object detection algorithm.

1. Constructing the Training Set

To train the model, you must define the target label y for every grid cell in the input image.

Setup

- **Classes:** Suppose we want to detect 3 classes: Pedestrians, Cars, Motorcycles.
- **Grid:** We use a 3×3 grid.
- **Anchors:** We use **2 anchor boxes**.
- **Vector Size:** Each bounding box requires 5 numbers (p_c, b_x, b_y, b_h, b_w) plus the number of classes (3).
 - Dimension per anchor: $5 + 3 = 8$.
 - Total dimension per grid cell (with 2 anchors): $2 \times 8 = 16$.

Output Volume Dimensions

The target output volume will be:

$Grid_Height \times Grid_Width \times (\text{Number of Anchors} \times (5 + \text{Number of Classes}))$

In this example: $3 \times 3 \times 16$.

Assigning Labels (y)

For each grid cell, the label vector y is constructed based on the objects present in the image.

- **Scenario A: Grid cell contains no object**
 - Target y : Both anchor boxes have $p_c = 0$. The rest are "don't cares."
 $y = [0, ?, ?, \dots, 0, ?, ?, \dots]$
- **Scenario B: Grid cell contains an object (e.g., a Car)**
 1. **Locate:** Find the grid cell containing the **midpoint** of the car.
 2. **Select Anchor:** Compare the shape of the car's ground truth bounding box to the two anchor box shapes.

- If Anchor Box 2 has a higher **IoU** with the car, assign the car to the *second* portion of the vector.

3. Construct Vector:

- Anchor 1 portion: $p_c = 0$ (unused).
- Anchor 2 portion: $p_c = 1$, followed by the car's bounding box coordinates and class label ($c_2 = 1$).

2. Making Predictions (Inference)

Once trained, the ConvNet performs object detection in a single forward pass.

- **Input:** An image (e.g., $100 \times 100 \times 3$).
- **Output:** The $3 \times 3 \times 16$ volume.
- **Interpretation:**
 - Each of the 9 grid cells predicts **2 bounding boxes** (one per anchor).
 - Total predictions: $9 \times 2 = 18$ bounding boxes.
 - Most of these boxes will have a very low probability ($p_c \approx 0$) because most grid cells do not contain objects.

3. Post-Processing: Non-Max Suppression (NMS)

The raw output contains many noisy bounding boxes and redundant detections. NMS is used to filter them.

1. **Discard Low Probability:** Remove any bounding box where the probability score (p_c) is below a certain threshold (e.g., < 0.5).
 2. **Class-Specific NMS:**
 - For the **Pedestrian** class: Run NMS to select the best boxes and suppress overlapping ones.
 - For the **Car** class: Run NMS independently.
 - For the **Motorcycle** class: Run NMS independently.
 3. **Final Output:** The remaining boxes after NMS constitute the final detections.
-

Region Proposals (R-CNN)

Motivation

The **Sliding Windows** approach is inefficient because it classifies thousands of windows where there are clearly no objects (e.g., empty sky or background). **Region Proposals** attempt to solve this by selecting only a few "interesting" regions to run the classifier on.

1. R-CNN (Regions with CNNs)

Proposed by Girshick et al., R-CNN replaces the exhaustive sliding window search with a selective search algorithm.

- **Step 1: Segmentation:** Run a segmentation algorithm to find "blobs" of texture or color that might be objects. This generates roughly 2,000 candidate regions.
- **Step 2: Classification:** Run a ConvNet classifier on **only** these 2,000 proposed regions to determine if they contain an object (Car, Pedestrian, etc.).
- **Step 3: Bounding Box Regression:** The network also outputs a corrected bounding box (b_x, b_y, b_h, b_w) to refine the rough proposal generated by the segmentation algorithm.

2. Iterations and Improvements

The original R-CNN was quite slow because it classified regions one at a time. This led to several faster iterations:

- **Fast R-CNN:** Uses a **convolutional implementation of sliding windows** (similar to the concept used in YOLO) to classify all proposed regions more efficiently. This fixed the classifier speed but left the region proposal step (segmentation) as a bottleneck.
- **Faster R-CNN:** Replaces the traditional segmentation algorithm with a **Convolutional Neural Network** to propose regions. This significantly speeds up the proposal step.

Comparison with YOLO

- **R-CNN Family:** Generally slower because it is a two-step process (1. Propose regions, 2. Classify regions).
- **YOLO (You Only Look Once):** Generally faster because it treats detection as a single regression problem in one evaluation.

- *Note:* While R-CNN is influential and widely cited, single-shot algorithms like YOLO are often preferred for real-time applications due to their speed.
-

Semantic Segmentation with U-Net

Evolution of Computer Vision Tasks

To understand Semantic Segmentation, it helps to see where it fits in the hierarchy of computer vision tasks:

1. Image Classification / Object Recognition:

- *Goal:* Determine *what* is in the picture.
- *Output:* A single class label (e.g., "Cat").

2. Object Detection:

- *Goal:* Find the object and its location.
- *Output:* A bounding box (b_x, b_y, b_h, b_w) around the object.

3. Semantic Segmentation:

- *Goal:* Draw a careful outline around the object to determine exactly **which pixels belong to the object** and which do not.
- *Output:* A pixel-by-pixel map where every single pixel is assigned a class label.

Motivation and Applications

Why do we need pixel-perfect accuracy?

1. Self-Driving Cars

- **Limitation of Bounding Boxes:** Drawing a bounding box around a winding road is messy and not very useful for navigation.
- **Segmentation Solution:** The algorithm labels every pixel as "drivable surface" or "not drivable." This tells the car exactly which irregular shapes on the ground are safe to drive on.

2. Medical Imaging

- **Chest X-Rays:** Segmenting organs (lungs, heart, clavicle) helps doctors spot irregularities and plan surgeries.
- **Brain MRI:** Automatically segmenting tumors saves radiologists time and assists in precise surgical planning.

The Problem Formulation

- **Input:** An image of size $H \times W \times C$ (e.g., Height, Width, RGB channels).
- **Target Output:** A matrix of size $H \times W$ (or $H \times W \times \text{num_classes}$).
 - Instead of a single label y for the whole image, we want a label $y_{i,j}$ for every pixel coordinate (i, j) .
 - *Example:* 0 = Background, 1 = Car, 2 = Building, 3 = Road.

The U-Net Architecture

Standard Convolutional Neural Networks (like AlexNet or ResNet) typically reduce the spatial dimensions (n_H, n_W) as the network goes deeper to summarize information into a single vector. Semantic segmentation requires the opposite at the end: we need to **output a full-resolution image**.

Architecture Flow

1. **Contraction (Encoder):** The first half of the network looks like a standard ConvNet. It applies convolutions and pooling to extract features, reducing height and width while increasing channels.
2. **Expansion (Decoder):** The second half of the network **increases** the height and width back to the original image size while decreasing channels.
 - This creates a U-shaped architecture (hence the name **U-Net**).

Key Operation: Expanding Dimensions

To go from a small feature map (e.g., 7×7) back to a large image (e.g., 14×14 and eventually the original size), we need a specific operation that performs the reverse of a convolution/pooling. This is called the **Transpose Convolution**.

Transpose Convolutions

Purpose

The Transpose Convolution is a key building block of the **U-Net** architecture. Its primary function is **Upsampling**: increasing the spatial dimensions (height and width) of a volume.

- **Normal Convolution**: typically reduces dimensions (e.g., $6 \times 6 \rightarrow 4 \times 4$).
- **Transpose Convolution**: increases dimensions (e.g., $2 \times 2 \rightarrow 4 \times 4$).

How It Works

Unlike a normal convolution, where you place a filter on the *input* and calculate a sum-product to get a single pixel, a transpose convolution takes a single pixel from the *input* and multiplies it by the filter to project values onto a larger *output* region.

Detailed Example

Goal: Expand a 2×2 input to a 4×4 output.

Parameters:

- **Input**: 2×2
- **Filter (f)**: 3×3
- **Padding (p)**: 1 (This usually implies cropping the border of the result).
- **Stride (s)**: 2

The Process:

1. **Select Input Pixel**: Take the upper-left pixel of the input (value = 2).
2. **Multiply & Project**: Multiply the entire 3×3 filter by this scalar value (2).
3. **Place on Output**: Place the resulting 3×3 block onto the output grid at the corresponding position.
4. **Shift by Stride**: Move to the next input pixel (top-right, value = 1).
 - Multiply the filter by 1.
 - Place this new 3×3 block on the output grid, shifted to the right by the **stride** (2 pixels).
5. **Handle Overlap**:
 - Because the stride (2) is smaller than the filter size (3), the projected blocks will overlap.

- **Crucial Rule:** Where the projected blocks overlap, **add the values together**.
6. **Repeat:** Continue this for all input pixels, shifting down by the stride for the next row.

Result

The final output is a larger matrix (e.g., 4×4) composed of the summed projections of the filter weighted by the input pixels.

- *Note:* There are multiple ways to upsample (e.g., nearest neighbor interpolation), but Transpose Convolution is preferred in deep learning because the filter weights are **learnable parameters**, allowing the network to learn the optimal way to upsample the features.
-

U-Net Architecture Intuition

High-Level Overview

The U-Net architecture is designed for semantic segmentation and consists of two main paths:

1. Contraction Path (Encoder):

- Uses **normal convolutions**.
- **Compresses** the image: Reduces height and width while increasing depth (channels).
- **Result:** Captures **high-level contextual information** (e.g., "there is a cat in the lower-right area") but loses detailed spatial information due to downsampling.

2. Expansion Path (Decoder):

- Uses **transpose convolutions**.
- **Expands** the representation: Increases height and width back to the original input size.

The Key Innovation: Skip Connections

To improve precision, U-Net adds **skip connections** that copy activation blocks from the earlier layers (encoder) directly to the corresponding later layers (decoder).

Why Skip Connections are Necessary

The final layers need to make a pixel-perfect decision ("Is *this* specific pixel part of a cat?"). To do this, they need two types of information:

1. **Context (from deep layers):** "There is a cat-like object generally in this area." (Low spatial resolution).
2. **Fine Detail (from early layers):** "There is fur texture or an edge at this exact pixel coordinate." (High spatial resolution).

The Mechanism:

- Since the deep layers have low spatial resolution, they lack the fine-grained detail needed to draw sharp outlines.
 - The skip connections inject high-resolution, low-level features (texture, edges) from the encoder directly into the decoder.
 - This allows the network to combine **context** (what object is this?) with **texture/localization** (where exactly are its edges?) to generate a precise segmentation map.
-

U-Net Architecture Details

Origin

The **U-Net** architecture was developed by Olaf Ronneberger, Philipp Fischer, and Thomas Brox.

- **Original Application:** Biomedical image segmentation (e.g., segmenting cells or organs).
- **Current Usage:** Foundational architecture for many computer vision semantic segmentation tasks beyond medical imaging.

Architecture Walkthrough

The name "U-Net" comes from the U-shape of the diagram when drawn out. It consists of a **Contraction Path (Encoder)** on the left and an **Expansion Path (Decoder)** on the right.

1. Contraction Path (The "Down" Side)

This section functions like a standard convolutional neural network, capturing context and reducing spatial dimensions.

- **Input:** Image of size $h \times w \times 3$ (RGB).
- **Operations:**
 - **Conv Layers (Black Arrows):** Standard feed-forward convolutions followed by ReLU activations. These increase the number of channels (depth) while maintaining or slightly reducing spatial dimensions depending on padding.
 - **Max Pooling (Red/Down Arrows):** Reduces the height and width (h, w) to downsample the feature map.
- **Result:** A deep, low-resolution feature map that contains high-level contextual information ("what" is in the image).

2. Expansion Path (The "Up" Side)

This section increases the spatial dimensions back to the original input size to generate the precise segmentation map.

- **Operations:**
 - **Transpose Convolution (Green Arrows):** Upsamples the feature map, increasing height and width while typically reducing the channel depth.
 - **Skip Connections (Grey Arrows): Crucial Step.**
 - Copy the high-resolution activation maps from the *contraction path* (left side).
 - Concatenate them with the upsampled features from the *expansion path* (right side).
 - *Purpose:* Re-introduces fine-grained spatial detail lost during max pooling.
 - **Standard Convolutions:** After the concatenation, standard convolutions merge the context (from the upsampling) and the detail (from the skip connection).

3. Final Output

- **Last Layers:** After the final upsampling and skip connection, the spatial dimensions match the original input ($h \times w$).
- **1x1 Convolution (Magenta Arrow):** Maps the final feature vector to the desired number of classes.
- **Output Dimensions:** $h \times w \times n_{classes}$

- For every pixel (i, j) , there is a vector of size $n_{classes}$ representing the probability of that pixel belonging to each class.
 - **Segmentation Map:** Apply argmax over the $n_{classes}$ dimension to assign a single label to each pixel.
-